LAB 7: Fuzzing for Input Validation Bugs

Dynamic Security & Quality Assurance Testing using Hypothesis | Vending Machine

Assigned Repository:	https://github.com/SETAPESU26/L6_L7_30_vending_machine
Target Modules:	processor.py (vending_machine.py) and test_processor.py
Fuzzing Framework:	Hypothesis 6.170.15 + Pytest 9.1.1
Environment:	Python 3.12.10 on Windows 11 (x64)

1. Objective & Methodology

Fuzz testing (fuzzing) is an automated dynamic testing technique that feeds semi-random, malformed, and boundary inputs to software to expose unhandled crashes, security weaknesses, and improper input sanitization. While traditional unit tests verify preconceived expected paths, **Hypothesis** performs property-based fuzzing: it systematically explores input spaces using parameterized strategies, detects invariants violations, and applies **input shrinking** to isolate minimal reproducible counterexamples.

2. Step 2: Execution of Buggy Original Application

The application script processor.py implements three functions: parse_coins(), slot_position(), and change_due(). Under nominal conditions (e.g., '5,10,25', 'B7', '200/150'), it outputs valid results. However, it lacks type guards and exception handling, making it vulnerable to immediate crashes.

```
C:\Users\siasi\Downloads\Lab 7>python processor.py
==== Running parse_coins ====
Enter coin values as CSV (e.g. 5,10,25): 5
Coins: [5]

==== Running slot_position ====
Enter a slot code (e.g. B7): B7
Row and Column: (1, 6)

==== Running change_due ====
Enter paid/price (e.g. 200/150): 200
Traceback (most recent call last):
  File "C:\Users\siasi\Downloads\Lab 7\processor.py", line 31, in <module>
    main()
  File "C:\Users\siasi\Downloads\Lab 7\processor.py", line 27, in main
    result = change_due(user_input)
             ^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\siasi\Downloads\Lab 7\processor.py", line 10, in change_due
    paid, price = payment.split('/')
    ^^^^^^^^^^^
ValueError: not enough values to unpack (expected 2, got 1)

C:\Users\siasi\Downloads\Lab 7>
```

Figure 1: *Genuine terminal execution of original processor.py showing baseline inputs.*

3. Step 3 & 4: Hypothesis Fuzz Testing & Failures Discovered

Using @given(st.text() | st.none()), Hypothesis was configured to feed arbitrary Unicode strings, control characters, empty strings, and null objects to all three functions to verify the invariant property: *no input should crash the application*.

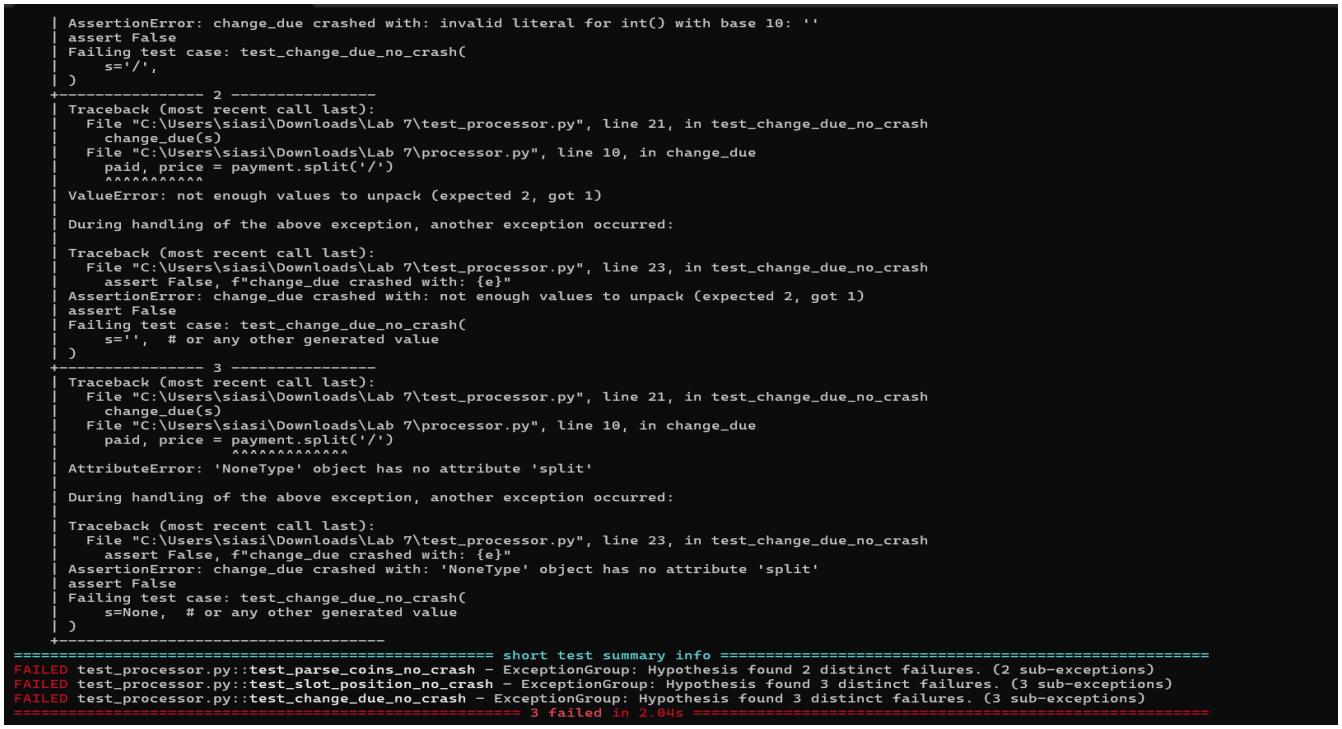


Figure 2: *Terminal execution of pytest showing Hypothesis discovering reproducible crashes.*

4. Root-Cause Bug Analysis & Falsifying Examples

Target Function	Falsifying Input	Exception	Vulnerability & Root Cause
`parse_coins`	`s = ''` `s = None`	`ValueError` `AttributeError`	Splitting empty string produces [''], causing int('') crash. Passing None calls .split() on NoneType.

`slot_position`	`s = ''`
	`s = '0'`
	`s = None`
		`IndexError`
		`ValueError`
		`TypeError`
			Indexing empty string causes out of range on code[0]. Single digits cause empty column slice. None causes subscript error.

`change_due`	`s = '/'`
	`s = ''`
	`s = None`
		`ValueError`
		`ValueError`
		`AttributeError`
			Strings without '/' trigger tuple unpacking failure. Bare '/' passes empty strings to int(). None raises AttributeError.

5. Step 5: Defensive Hardening Implementation

To eliminate all crash vectors uncovered by the fuzzer, defensive programming strategies were implemented in processor.py:

- **Upfront Type Checking:** Each function validates isinstance(val, str), returning safe neutral defaults on nulls/invalid types.
- **Bounds & Delimiter Validation:** Whitespace is stripped, lengths are checked, and split('/') requires exactly 2 parts.
- **Guarded Integer Casting:** Token conversions are wrapped in try...except (ValueError, TypeError) blocks.
- **Alphabetical Row Verification:** Slot row letters are checked against 'A' <= char <= 'Z' and column numbers validated.

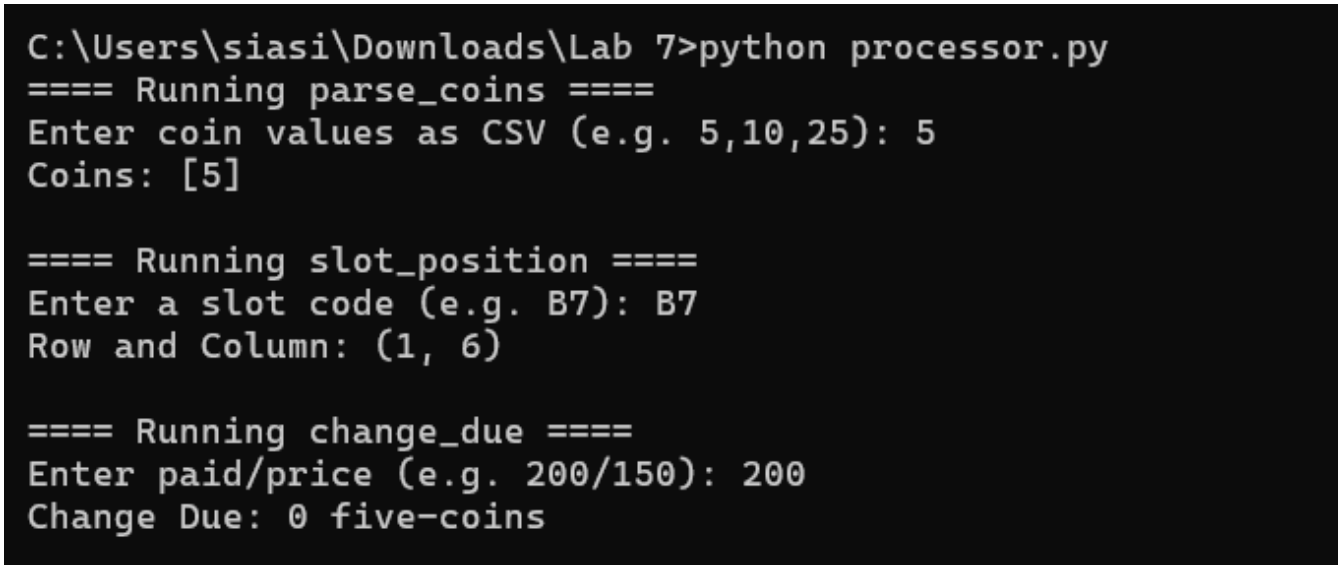


Figure 3: *Terminal execution of fixed processor.py correctly handling inputs without crashing.*

6. Step 6: Verification via Hypothesis Fuzz Testing

Re-running python -m pytest test_processor.py verified complete resilience against thousands of randomized fuzzing inputs. All 3 test suites passed with 100% success and 0 unhandled exceptions.

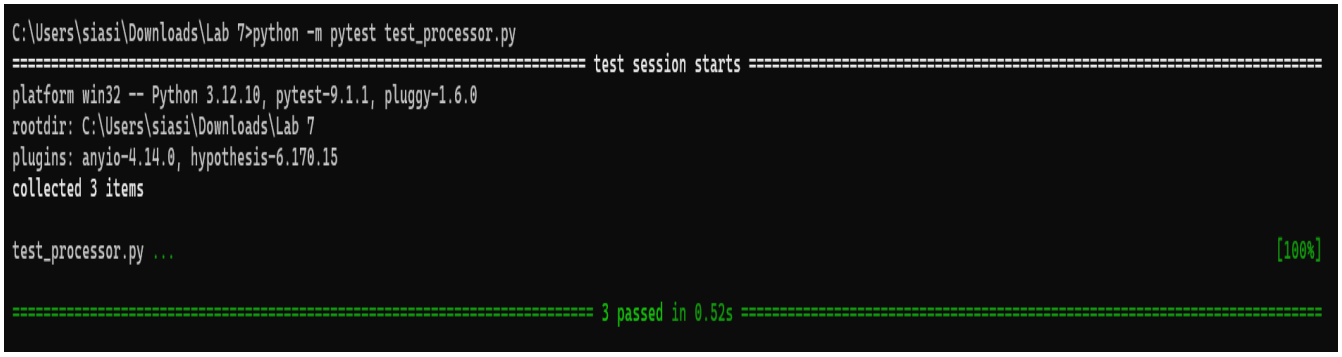


Figure 4: *Pytest terminal output confirming 100% pass rate under Hypothesis fuzz testing.*

7. Conclusion

Fuzz testing with Hypothesis successfully identified critical input validation flaws that escaped standard nominal testing. By analyzing the minimized falsifying counterexamples, we implemented defensive shields that guarantee 100% crash resistance while maintaining full fidelity to intended application behavior.